UNIVERSITY OF WOLLONGONG IN DUBAI

Assignment Cover Sheet

Subject Code: CSCI 316
Subject Name: Big Data Mining Techniques and Implementation
Submission Type:
Assignment Title:
Student Name:
Muhammad Iqbal 8260308
Alina Khan 8923292
Riya Jain 8891333
Mohammad Hasheer Choudhary 7832230
Khaleelullah Syed 7929419
Pavel Bashkirev 8204512
Mostafa Shalash 7391493
Mohammed Amer 8424342
Danish Tanwar 8551248
Magamed Khassanov 8488265
Student Number:
Student Phone/Mobile No.
Student E-mail: @uowmail.edu.au
Lecturer Name:
Due Date:
Date Submitted:

PLAGIARISM:	**DECLARATION:**
The penalty for deliberate plagiarism is FAILURE in the subject. Plagiarism is cheating by using the written ideas or submitted work of someone else. UOWD has a strong policy against plagiarism. The University of Wollongong in Dubai also endorses a policy of non-discriminatory language practice and presentation. **PLEASE NOTE:** STUDENTS MUST RETAIN A COPY OF ANY WORK SUBMITTED	I/We certify that this is entirely my/our own work, except where I/we have given fully-documented references to the work of others, and that the material contained in this document has not previously been submitted for assessment in any formal course of study. I/we understand the definition and consequences of plagiarism. **Signature of Student:**

Optional Marks:

Comments:

Lecturer Assignment Receipt (To be filled in by student and retained by Lecturer upon return of assignment)

Subject: **Assignment Title:**
Student Name: **Student Number:**
Due Date: **Date Submitted:**
Signature of Student:

Student Assignment Receipt (To be filled in and retained by Student upon submission of assignment)

Subject: **Assignment Title:**
Student Name: **Student Number:**
Due Date: **Date Submitted:**
Signature of Lecturer

Table of contents

1. Introduction....3
2. Problem Statement....3
3. Dataset Description....3
4. Big Data Justification and Tools....4
5. Methodology....4
6. Results....7
7. Discussion and Interpretation....9
8. Conclusion and Future Work....10
References....10
Appendix A....11
Appendix B....25
Appendix C....33
Appendix D....46

1. Introduction

The rapid growth of urban transportation systems has led to the generation of large-scale mobility data. Efficient analysis of such data can provide valuable insights into travel patterns, operational efficiency and urban planning. One important predictive task in this domain is estimating the duration of taxi trips based on trip-related attributes.

This project focuses on predicting taxi trip duration using large-scale New York City taxi trip data. Due to the volume and complexity of the dataset, distributed data processing and machine learning techniques are required. Apache Spark is used to process and model the data efficiently. The project applies supervised regression techniques and evaluates both baseline and ensemble models to assess predictive performance.

---

2. Problem Statement

The objective of this project is to build a supervised machine learning model that predicts the duration of a taxi trip in minutes based on temporal, spatial and transactional features.

Formally, this is defined as a regression problem where the target variable is continuous. The project also aims to evaluate whether ensemble learning methods can improve predictive accuracy and stability compared to single baseline models.

---

3. Dataset Description

The dataset used in this project is the New York City Taxi Trip Record Data, published by the NYC Taxi and Limousine Commission (TLC). Each record represents an individual taxi trip and includes information such as pickup and drop-off timestamps, trip distance, passenger count, fare components and pickup/drop-off locations.

Key characteristics of the dataset include:

- Very large size (millions of records)
- Mix of numerical, categorical and temporal features
- Real-world noise and outliers

The dataset is stored in Parquet format, a columnar storage format optimized for large-scale analytics.

---

4. Big Data Justification and Tools

Due to the size and complexity of the dataset, traditional single-machine tools such as Pandas are not suitable. Apache Spark was selected for this project because it:

- Supports distributed data processing
- Efficiently handles large datasets
- Integrates machine learning pipelines via Spark MLlib
- Natively supports Parquet files

Spark enables scalable preprocessing, feature engineering and model training, making it well-suited for this problem.

---

5. Methodology

In this part, we will discuss the methodologies we used in our preprocessing and implementation and the reasoning behind our model selection and feature engineering.

5.1 **Data Preprocessing and Feature Engineering**

Data preprocessing was the critical first step to transform raw, noisy taxi data into a clean format suitable for machine learning. We wrangled our data manually and we also performed some feature engineering to see what features our models may prefer.

Raw sensor data often contains errors, which is why we started with manually cleaning the data using domain knowledge, visualizations and facts that we found in our Exploratory Data Analysis (*eda.ipynb snippets available in Appendix A*). Multiple strict filtering rules were applied using Spark SQL to remove records that would confuse the model:

- Created a new feature called `trip_duration_min` using the timestamp columns, which tells us the duration of the trip in minutes.
- Trips with negative durations or those lasting less than one minute were removed, as they likely represented system errors or immediate cancellations.
- We filtered out records with zero passenger counts or zero trip distances.
- Extreme outliers (e.g., trips costing over $500 or lasting more than 5 hours) were trimmed to prevent them from skewing the regression model.
- We removed NaN values, some of which were specified in the NYC Taxi site, like `RatecodeID` of 99.0 actually being a NaN value.
- The NYC Taxi site says that it doesn't record cash tip amounts, so we removed cash rides and some more as specified in our source.

- We looked at correlation between `fare_amount` and distance, duration and tip columns to make sure that the tipping is not simply percentage based and that fare amount. Technically, fare amount calculation contains information about distance and duration, however, we decided to leave them in.

After this main cleaning, we also decided to do some feature engineering using L1 regularization and Decision Tree feature importance. We found out that the features that both models chose were fairly different. Considering that our feature space is not too high, we decided that it would be appropriate to use all of our features for the model training. This also helps us ensure that our Decision Tree and Linear Regression can find linear and non-linear relationships themselves and utilize them for prediction. Furthermore, we decided against using PCA, to make sure that our results will remain interpretable.

5.2 **Model Selection**

Two baseline regression models and one ensemble model were selected:

5.2.1 **Linear Regression**

Linear Regression was used as an initial baseline model due to its simplicity and interpretability. However, it assumes linear relationships between features and the target variable.

5.2.2 **Decision Tree Regression**

A Decision Tree Regressor was used as a non-linear baseline model. Decision trees are capable of modeling complex, non-linear relationships and are robust to skewed data distributions, making them suitable for real-world transportation data.

5.2.3 **Bagging Ensemble using Decision Trees**

To further improve predictive performance and model stability, an ensemble learning approach was implemented using Bagging (Bootstrap Aggregating). As per the project requirements, this was implemented from scratch rather than using a pre-built library function.

5.3 **Model Implementation Details**

Implementation details about the pipelines, hyperparameters, manual functions and after-split preprocessing (*snippets available in Appendix C*):

5.3.1 **Model Preprocessing**

The dataset containing all the features and parquet files was split for training and testing purposes. Seed of 42 was used for reproducibility, however, even with the same seed, the differences in cores and partitions number may produce different results. Train and test sets were cached for efficient memory usage.

For our implementation of all the models, we needed reliable pipelines. We used Spark's `Pipeline` class for our linear regression and decision tree regressor. They both share the same initial preprocessing stages for categorical columns VendorID and RatecodeID (LocationID columns were not used to not explode the feature space). `StringIndexer` is used for converting categoricals to numerical indices, which are the inputs for the `OneHotEncoder`. Then all of the features are combined together for `VectorAssembler` to turn them into a single feature vector, which is required for SparkML.

An additional `StandardScaler` is applied to the linear regression pipeline to ensure stable results and because of the regularization we will be applying. Now, all we need to do is add the actual models to our pipelines.

5.3.2 **Linear Regression Implementation**

We used (regParam = 0.1) and (elasticNetParam = 0.5) to combine the advantages of both Lasso and Ridge regularization, which can remove the noisy or multicollinear features, as well as setting the penalization not too high or low for best generalization.

5.3.3 **Decision Tree Implementation**

For our Decision Tree we focused on only two hyperparameters: `maxBins` and `maxDepth`. The former is a Spark specific parameter that sets the number of splits it does on a range of a feature. For example, if we have Age ranging from 0 to 100 and maxBins=5, it will split the range into five categories. The latter is a general tree hyperparameter, which decides how deep from the root the decision tree needs to base its decisions. Our Decision Tree model was picked by using the cross validator and as the result the parameters are set as `(maxBins=64)` and `(maxDepth=10)`.

5.3.4 **Manual Cross Validator Implementation**

The cross validation is created as a function.

As its inputs it takes the train data, pipeline to use, parameter options (a dictionary of arrays), number of k folds and the seed. It explodes the `param_options` dictionary to make combinations of all parameters using `itertools.product` and stores it in the `param_grid`, which is an array of dictionaries.

Before the main loop, we index our data with random fold IDs and cache it since it will be reused throughout the function. We also create variables to store the RMSE accuracy metric, the best parameter dictionary and the best model.

We start our loop using the parameters in the parameter grid. `fold_scores` variable is instantiated to score the average RMSE for the parameters across the fold. Then we nest in another loop, which is our K-Fold cross validation, it uses the indexed frame to pick the training and validation folds at each iteration. The parameters for the decision trees are then set and we train our models as usual. We keep track of the best models and parameters in our main loop using the average scores and in the end we pick the best model. We train it on the input train data and return it as the final output.

5.3.5 **Bagging Ensemble Implementation**

The bagging process consists of two functions: `bootstrapping` and `aggregating_predictions`. Ensembling is done only using decision tree models.

Bootstrapping takes the train set, pipeline, number of trees in the ensemble and parameter dictionary as inputs. We cache the data set and start a loop on the number of trees. In this loop, we train the models as usual, but the training set is resampled at each iteration using `sample(withReplacement=True, fraction=1.0, seed=i+42)`, which retains the size of the set, but allows for recurring rows. Seeds are changed at every loop to make sure it is truly random. We store each model in the `models` array.

In the aggregation function, we take the models from the bootstrapping and our testing set. We index the data, cache it and then create a new frame called `final_df` to store only the IDs and our target feature. We then create a testing loop using our models array and we join the prediction results using IDs back to our `final_df` variable. After the loop, we create a new prediction column that is the average tip amount using every tree prediction. As the final output, we return a frame .

5.4 **Docker Containerization**

We used Docker to containerize our project for sharing (*code snippets available in Appendix D*). Docker ensures that our machine learning pipeline runs on different operating systems by leveraging Docker Core and Linux Subsystems. We built our final image by using the base python-slim image as our foundation. Java Runtime Environment was installed, as well as libraries specified in requirements.txt, that we created by running our project in a virtual environment.

---

6. Results

In this part, we will discuss the results of our work, using three chosen performance metrics: Root Mean Square Error (RMSE), Mean Absolute Error (MAE), R-squared (R2). R-squared was

calculated for both the training and testing set of the Linear Regression and Decision Tree. In the context of tipping amount:

- MAE explains how off in dollars are we per tip (MAE = 0.8 means $0.80)
- RMSE is similar to MAE, but it penalizes heavily on outliers, which increases its score
- R2 explains what percentage or ratio of tipping behavior can our model explain

Our main goal is to maximize the score of R2 and minimize MAE and RMSE. However, considering that tipping can be quite sporadic and can be based on external factors like driver behavior, cleanliness, outside environment, etc. We are not expecting extremely good results from the features that we are given.

Note: The scores that we show here are a result from setting our Spark Cores to 8GB and our Spark Driver Memory to 4GB. Results may vary even with the same seeds, but the ranking of each and every model remains the same every time.

6.1 **Linear Regression Results**

RMSE	MAE	R2 (of test)	R2 (of train)
2.393	1.276	0.604	0.601

Overall satisfactory scores, it seems like linear regression was able to capture real linear relationships in the dataset. The R-squared of test and train are very close, so the model was able to generalize well.

6.2 **Manual Cross Validation and Decision Tree Results**

RMSE	MAE	R2 (of test)	R2 (of train)
2.374	1.260	0.610	0.619

Once again - good scores. However, they are slightly better than the linear regression, which may signify that there are non-linear relationships in the data that linear regression didn't utilize in the prediction. The scores are used by training a decision tree with `(maxBins=64)` and `(maxDepth=10)`. Interestingly, at `(maxDepth=15)`, the scores got considerably worse.

6.3 **Manual Bagging Ensemble Results**

RMSE	MAE	R2 (of test)
2.375	1.258	0.617

Marginally, but we got the best scores with our bagging model. While it may seem underwhelming, the bigger advantage of the ensemble process is that it reduces overfitting, improves generalization and keeps scores fairly stable.

---

7. Discussion and Interpretation

The **Linear Regression** model served as a baseline but ever so slightly underperformed compared to tree-based methods. This suggests that the relationship between taxi trip features (like location, time of day) and the target variable (tip amount) may be non-linear. A simple straight-line equation cannot capture the complex behavioral patterns of tipping. However, if we apply the Grid Search cross validation on the Linear Regression, the scale may actually tip towards it.

The **Cross Validator** may have played a major role in the Decision Tree success. Even more improtantly, it allowed us to see that the scoring on resampled sets (folds) were not too different between each other, which suggests that we have not overfitted our model.

The **Decision Tree Regressor** improved every performance score. By splitting data based on conditions (e.g., "Is the trip longer than 5 miles?"), the tree successfully captured non-linear interactions. For instance, short trips might have a completely different tipping culture compared to long airport runs, which a single linear model misses but a decision tree handles well.

The **Manual Bagging Ensemble** yielded the best performance scores. Although the numerical increase in accuracy seems small, the reduction in RMSE demonstrates that averaging multiple trees successfully reduced the variance.

The use of **Apache Spark** was critical. Processing too many records involved heavy transformations (joins, aggregations) that would have caused memory failures (OOM) on standard Python/Pandas environments. The `Safe Mode` configuration and Parquet columnar storage ensured the pipeline ran efficiently within the 2GB/4GB memory constraints of the local machine.

Our hypothesis is that the reason why most of our models performed strikingly similar to each and have not broken through the consistent scoring ceiling is because of the nature of our data. Tipping behavior is sporadic and very human in nature. Using only ride details, we can only

predict so much, that is to say, we believe that we may have almost reached the predictive power of our features.

---

8. Conclusion and Future Work

Conclusion

This project successfully implemented a scalable machine learning pipeline to predict taxi trip tip amounts using Apache Spark. The results demonstrate that **non-linear approaches (Decision Trees)** may outperform linear baselines for prediction of tipping behavior using only ride details. Furthermore, implementing a **Bagging Ensemble from scratch** proved that aggregating weak learners reduces error variance, validating the theoretical benefits of ensemble learning. The project also highlighted the necessity of distributed computing tools like Spark for handling datasets exceeding 10 million records.

Future Work

- **New Features:** We want to see whether or not adding more subjective features in our dataset (e.g. ride ratings, weather conditions, driver ratings) would improve our score. If not, then this would mean that we need to do more extensive work on our existing dataset.
- **Hyperparameter Tuning:** Due to computational constraints, the Grid Search was limited to `maxDepth` and `maxBins`. Future iterations could explore `minInstancesPerNode` to further prune trees.
- **Advanced Ensembles:** While Bagging reduced variance, implementing **Boosting** (e.g., Gradient Boosted Trees) could effectively reduce bias and potentially yield higher accuracy.
- **External Data Integration:** Integrating weather data or traffic density reports could provide additional predictive signals that are currently missing from the dataset.

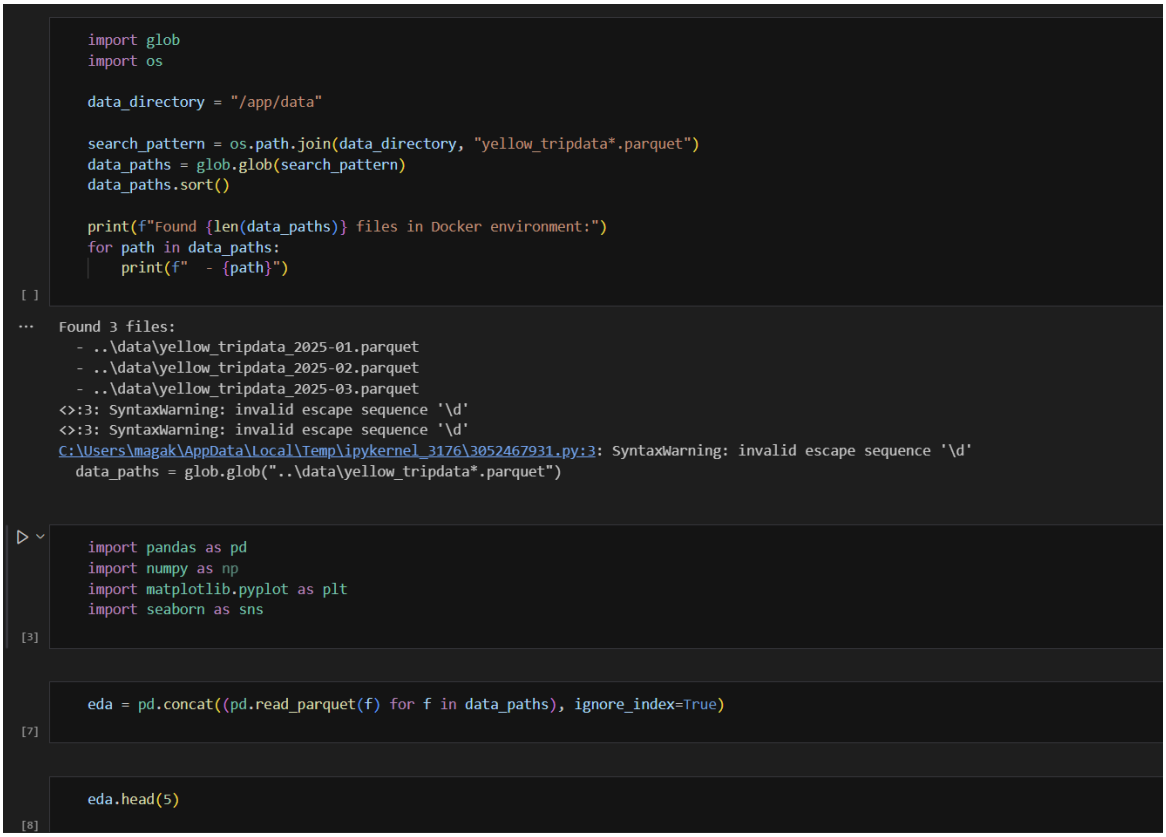
References

1. NYC Taxi and Limousine Commission (2025) *TLC Trip Record Data*. Available at: https://www.nyc.gov/site/tlc/about/tlc-trip-record-data.page (Accessed: 09 February 2026).
2. Zaharia, M. et al. (2016) Apache Spark: a unified engine for big data processing, *Communications of the ACM*, 59(11), pp. 56–65.
3. Breiman, L. (1996) 'Bagging predictors', *Machine Learning*, 24(2), pp. 123–140.

Link to the video presentation: https://youtu.be/OkyIILsT6TQ

Appendix A

This appendix contains snippets of the Exploratory Data Analysis (eda.ipynb)



```
[ ]
import glob
import os

data_directory = "/app/data"

search_pattern = os.path.join(data_directory, "yellow_tripdata*.parquet")
data_paths = glob.glob(search_pattern)
data_paths.sort()

print(f"Found {len(data_paths)} files in Docker environment:")
for path in data_paths:
    print(f"  - {path}")
```

```
...
Found 3 files:
  - ..\data\yellow_tripdata_2025-01.parquet
  - ..\data\yellow_tripdata_2025-02.parquet
  - ..\data\yellow_tripdata_2025-03.parquet
<>:3: SyntaxWarning: invalid escape sequence '\d'
<>:3: SyntaxWarning: invalid escape sequence '\d'
C:\Users\magak\AppData\Local\Temp\ipykernel_3176\3052467931.py:3: SyntaxWarning: invalid escape sequence '\d'
  data_paths = glob.glob("..\data\yellow_tripdata*.parquet")
```

```
[3]
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns
```

```
[7]
eda = pd.concat((pd.read_parquet(f) for f in data_paths), ignore_index=True)
```

```
[8]
eda.head(5)
```

Importing libraries, finding files paths and combining them in one dataframe

```
# Basic info
print(eda.shape)
print(eda.info())
```

```
(11198026, 20)
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 11198026 entries, 0 to 11198025
Data columns (total 20 columns):
 #   Column                 Dtype
---  ------                 -----
 0   VendorID               int32
 1   tpep_pickup_datetime   datetime64[us]
 2   tpep_dropoff_datetime  datetime64[us]
 3   passenger_count        float64
 4   trip_distance          float64
 5   RatecodeID             float64
 6   store_and_fwd_flag     object
 7   PULocationID           int32
 8   DOLocationID           int32
 9   payment_type           int64
 10  fare_amount            float64
 11  extra                  float64
 12  mta_tax                float64
 13  tip_amount             float64
 14  tolls_amount           float64
 15  improvement_surcharge  float64
 16  total_amount           float64
 17  congestion_surcharge   float64
 18  Airport_fee            float64
 19  cbd_congestion_fee     float64
dtypes: datetime64[us](2), float64(13), int32(3), int64(1), object(1)
memory usage: 1.5+ GB
None
```

Information about the frame

```
    missing_mask = eda.isna()
    print(missing_mask.sum())

    print("-"*60)
    missing_pct = (missing_mask.sum() / len(eda)) * 100
    print(missing_pct[missing_pct > 0])
```

```
VendorID                      0
tpep_pickup_datetime          0
tpep_dropoff_datetime         0
passenger_count         2263749
trip_distance                 0
RatecodeID              2263749
store_and_fwd_flag      2263749
PULocationID                  0
DOLocationID                  0
payment_type                  0
fare_amount                   0
extra                         0
mta_tax                       0
tip_amount                    0
tolls_amount                  0
improvement_surcharge         0
total_amount                  0
congestion_surcharge    2263749
Airport_fee             2263749
cbd_congestion_fee            0
dtype: int64
------------------------------------------------------------
passenger_count         20.215608
RatecodeID              20.215608
store_and_fwd_flag      20.215608
congestion_surcharge    20.215608
Airport_fee             20.215608
dtype: float64
```

Missing values (absolute and percentages)

```
cols_with_missing = eda.columns[eda.isna().any()].tolist()

all_same_rows = (missing_mask[cols_with_missing].sum(axis=1) == len(cols_with_missing)).sum()

if all_same_rows == eda.isna().any(axis=1).sum():
    print("Missing values occur in the same rows")
else:
    print("Missing values are in different rows")
```

[11]

```
Missing values occur in the same rows
```

```
missing_rows = eda[missing_mask.any(axis=1)]
missing_rows.head()
```

[12]

	VendorID	tpep_pickup_datetime	tpep_dropoff_datetime	passenger_count	trip_distance	RatecodeID	store_and_fwd_flag	PULocationID	DOLocationID	payment_
2935077	2	2025-01-01 00:39:17	2025-01-01 01:10:42	NaN	6.75	NaN	None	48	42	
2935078	2	2025-01-01 00:33:40	2025-01-01 01:10:34	NaN	3.88	NaN	None	4	48	
2935079	1	2025-01-01 00:09:32	2025-01-01 00:12:38	NaN	0.00	NaN	None	158	158	
2935080	2	2025-01-01 00:21:42	2025-01-01 00:36:29	NaN	3.55	NaN	None	137	211	
2935081	2	2025-01-01 00:56:04	2025-01-01 01:06:11	NaN	1.55	NaN	None	140	233	

Making sure that the NaN values occur in same rows, so we can delete them whole

```
len(eda[eda['trip_distance'] == 0]) / len(eda) * 100 # Remove error data
```

[13]

```
2.628909773919082
```

```
len(eda[eda['payment_type'] == 2]) / len(eda) * 100 # We need to remove cash payment type since it does not include tips
```

[14]

```
10.142475111238355
```

```
len(eda[eda['fare_amount'] < 0]) / len(eda) * 100 # Negative fare amount is also error data
```

[15]

```
4.782066053427632
```

Percentage of zero/”cash”/negative values in three features



```
sns.set_style("whitegrid")

numerical_cols = [
    "passenger_count",
    "trip_distance",
    "fare_amount",
    "extra",
    "mta_tax",
    "tip_amount",
    "tolls_amount",
    "improvement_surcharge",
    "total_amount",
    "congestion_surcharge",
    "Airport_fee",
    "cbd_congestion_fee"
]

categorical_cols = [
    "VendorID",
    "RatecodeID",
    "store_and_fwd_flag",
    "PULocationID",
    "DOLocationID",
    "payment_type"
]

# Print categorical distributions
for col in categorical_cols:
    print(f"\n{col}:")
    print(eda[col].value_counts())
```

[16]

Categorical features analysis

```
VendorID:
VendorID
2    8826711
1    2343055
7      27107
6       1153
Name: count, dtype: int64

RatecodeID:
RatecodeID
1.0     8391053
2.0      288602
99.0     125083
5.0       81032
3.0       27178
4.0       21318
6.0          11
Name: count, dtype: int64

store_and_fwd_flag:
store_and_fwd_flag
N    8891246
Y      43031
Name: count, dtype: int64

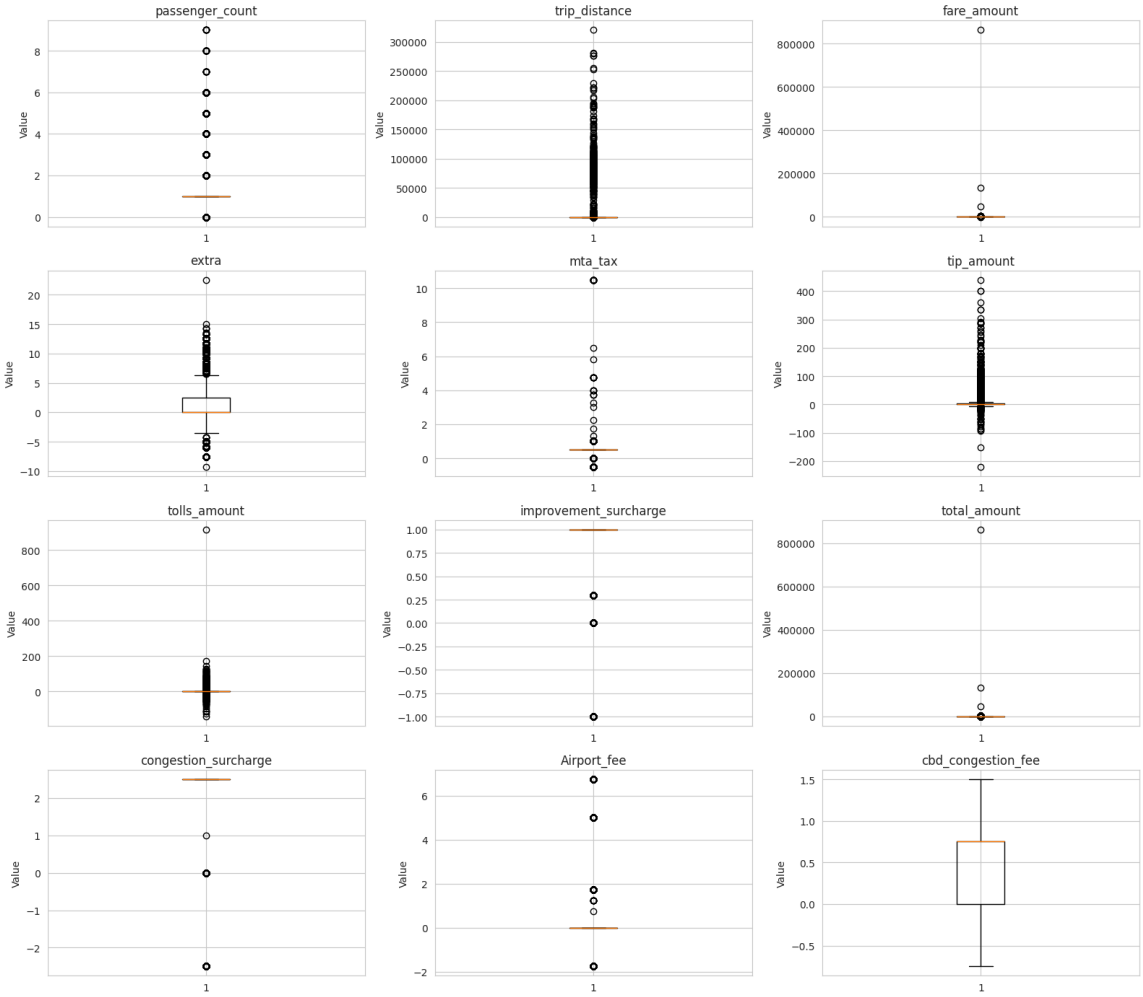
PULocationID:
PULocationID
161    513560
237    491957
236    456758
132    433506
230    377216
        ...
84          7
5           5
199         5
204         5
99          1
Name: count, Length: 261, dtype: int64
```

Snippet output of the code for categorical feature analysis

```
17]
# Box plots for outlier detection
fig, axes = plt.subplots(4, 3, figsize=(16, 14))
axes = axes.flatten()

for idx, col in enumerate(numerical_cols):
    axes[idx].boxplot(eda[col].dropna(), vert=True)
    axes[idx].set_title(col)
    axes[idx].set_ylabel('Value')
plt.tight_layout()
plt.show()
```

Boxplots code



Boxplots output || We can see that most numeric features are very outliers heavy, which will destroy our linear models if not addressed

As we can see from the boxplots, most features have a lot of outliers considerably skewing the data. Normalization should be applied - we can use RobustScaler for this.

```
[18]
from sklearn.preprocessing import RobustScaler

scale = RobustScaler()

eda_no_cash = eda[eda['payment_type'] != 2]
numeric_cleaned = eda_no_cash[numerical_cols].dropna()
numeric_cleaned = numeric_cleaned[numeric_cleaned['trip_distance'] != 0]
scaled_numerical_features = scale.fit_transform(numeric_cleaned)
```

Python

```
[19]
len(eda_no_cash[eda_no_cash['tip_amount'] > 0]) / len(eda_no_cash) * 100
```

Python

```
71.76241263277696
```

```
[20]
eda_no_cash['tip_amount'].describe()
```

Python

	tip_amount
count	1.006227e+07
mean	3.170271e+00
std	3.841287e+00
min	-2.200000e+02
25%	0.000000e+00
50%	2.610000e+00
75%	4.070000e+00
max	4.400000e+02

Robust scaling to see if it can meaningfully improve the outlier situation. Some more validation checks. We can also see that the non-zero positive tips make up 71% of our data, which may mean that the prediction for zeroes will be weak.

```python
# Check distribution of taxes and fees
fee_cols = [
    "mta_tax",
    "improvement_surcharge",
    "congestion_surcharge",
    "Airport_fee",
    "cbd_congestion_fee"
]

for col in fee_cols:
    print(f"\n{col}:")
    value_counts = eda[col].value_counts()
    percentages = (value_counts / len(eda) * 100).round(2)
    result = pd.DataFrame({'count': value_counts, 'percentage': percentages})
    print(result)
```

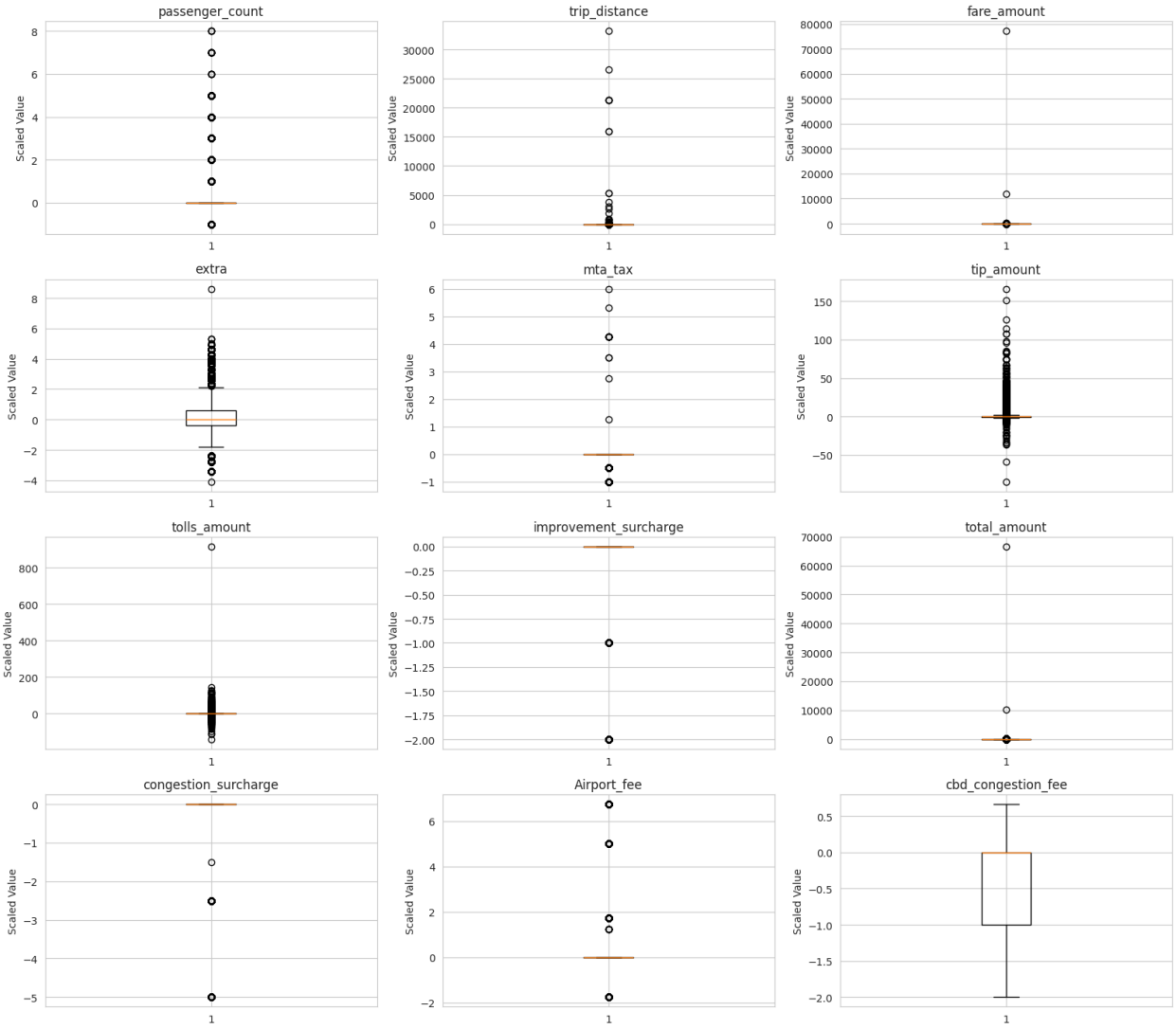
[21]

```
mta_tax:
            count  percentage
mta_tax
 0.50    10903553       97.37
-0.50      175826        1.57
 0.00      118446        1.06
 1.00         152        0.00
 10.50         24        0.00
 4.75          11        0.00
 4.00           4        0.00
 3.75           3        0.00
 6.50           1        0.00
 1.75           1        0.00
 5.80           1        0.00
 2.25           1        0.00
 1.32           1        0.00
 3.00           1        0.00
 3.25           1        0.00
```

Distribution of taxes and fees

```python
# Box plots for scaled numerical columns
fig, axes = plt.subplots(4, 3, figsize=(16, 14))
axes = axes.flatten()
for idx, col in enumerate(numerical_cols):
    axes[idx].boxplot(scaled_numerical_features[:, idx], vert=True)
    axes[idx].set_title(col)
    axes[idx].set_ylabel('Scaled Value')
plt.tight_layout()
plt.show()
```

Boxplots with robust scaled data code



Boxplots with robust scaled data output || Decent, but if we use RobustScaling we still need to manually clean the outliers first, which defeats a purpose slightly

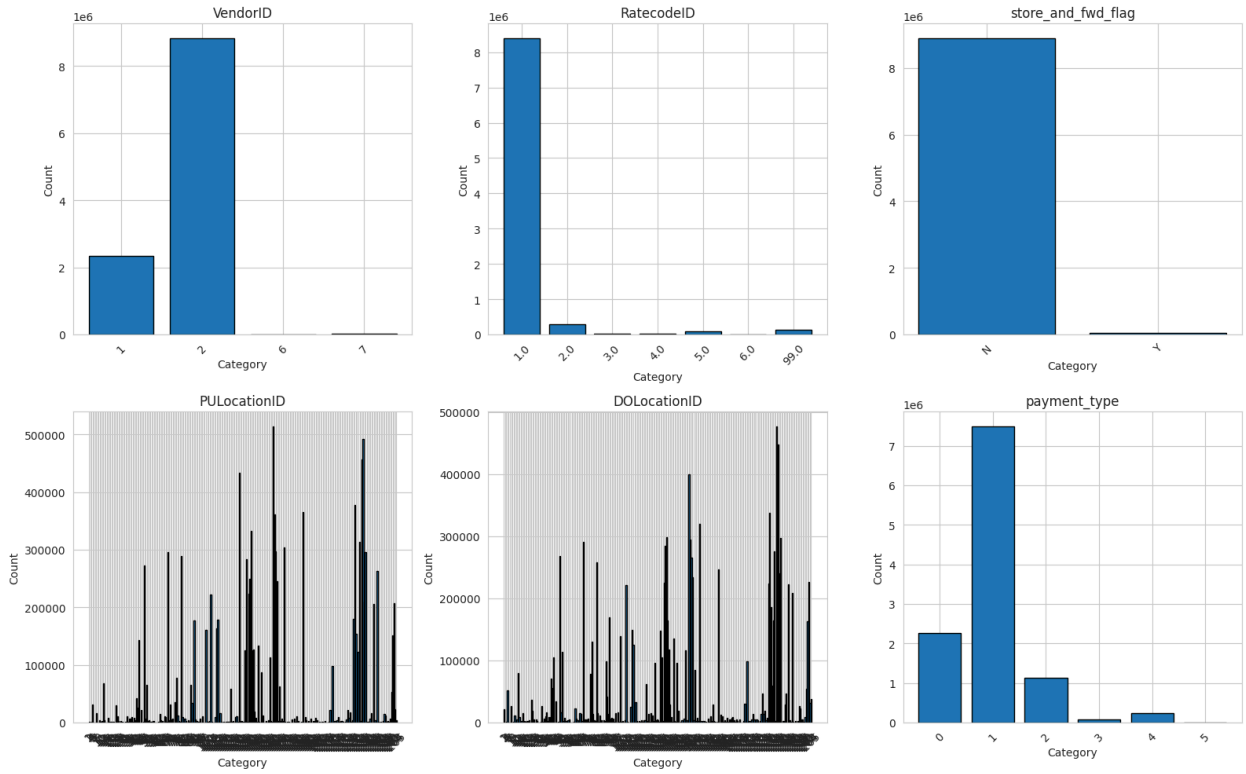
There are still some serious outliers in our data, even after applying RobustScaler. It might be best to just get rid of the outlier rows, for example trip distance being higher than around 900 is highly unlikely. Fare amount higher than 500 also unlikely

```python
# Bar charts for categorical columns
fig, axes = plt.subplots(2, 3, figsize=(16, 10))
axes = axes.flatten()

for idx, col in enumerate(categorical_cols):
    value_counts = eda[col].value_counts().sort_index()
    axes[idx].bar(value_counts.index.astype(str), value_counts.values, edgecolor='black')
    axes[idx].set_title(col)
    axes[idx].set_xlabel('Category')
    axes[idx].set_ylabel('Count')
    axes[idx].tick_params(axis='x', rotation=45)
plt.tight_layout()
plt.show()
```

[23] Python

Bar charts code



Bar charts plots output || Very big imbalances, but could be because we only used 3 months of the year.

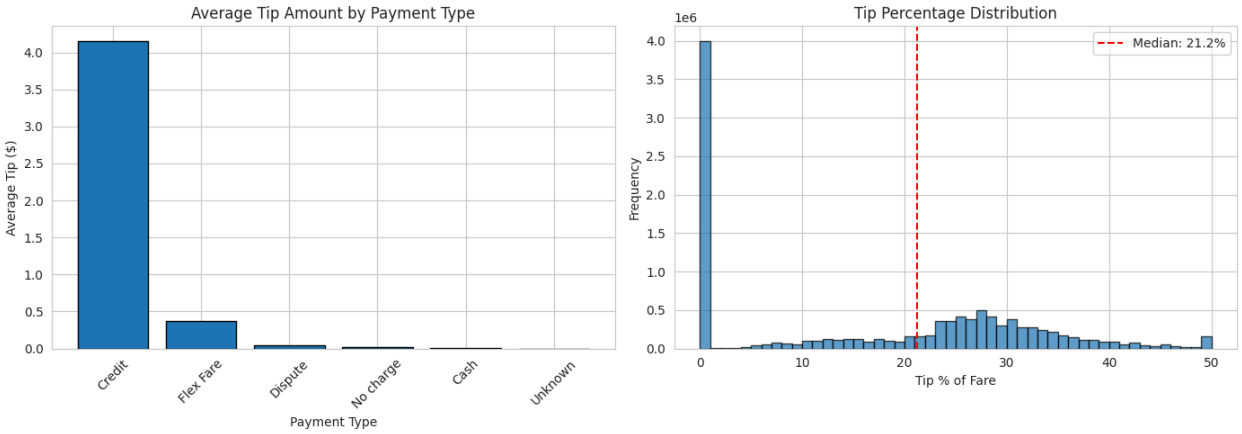
```python
# Tip amount analysis
fig, axes = plt.subplots(1, 2, figsize=(14, 5))

payment_labels = {0: 'Flex Fare', 1: 'Credit', 2: 'Cash', 3: 'No charge', 4: 'Dispute', 5: 'Unknown', 6: 'Voided'}
tip_by_payment = eda.groupby('payment_type')['tip_amount'].mean().sort_values(ascending=False)

axes[0].bar([payment_labels.get(x, str(x)) for x in tip_by_payment.index], tip_by_payment.values, edgecolor='black')
axes[0].set_title('Average Tip Amount by Payment Type')
axes[0].set_xlabel('Payment Type')
axes[0].set_ylabel('Average Tip ($)')
axes[0].tick_params(axis='x', rotation=45)

eda['tip_pct'] = (eda['tip_amount'] / eda['fare_amount'] * 100).clip(0, 50)
axes[1].hist(eda['tip_pct'].dropna(), bins=50, edgecolor='black', alpha=0.7)
axes[1].set_title('Tip Percentage Distribution')
axes[1].set_xlabel('Tip % of Fare')
axes[1].set_ylabel('Frequency')
axes[1].axvline(eda['tip_pct'].median(), color='red', linestyle='--', label=f"Median: {eda['tip_pct'].median():.1f}%")
axes[1].legend()
plt.tight_layout()
plt.show()
```

Bar charts for tip analysis code



Bar charts for tip analysis output || Mostly credit data is good for us. Most tips are close to zero (but it also includes other payment types). Usually tips are 20% of the fare amount (though because of the imbalance it is actually closer to around ~26-27%)

```python
# Datetime patterns
eda['pickup_hour'] = eda['tpep_pickup_datetime'].dt.hour
eda['pickup_day'] = eda['tpep_pickup_datetime'].dt.dayofweek
eda['pickup_month'] = eda['tpep_pickup_datetime'].dt.month

fig, axes = plt.subplots(1, 3, figsize=(16, 5))

hourly_trips = eda.groupby('pickup_hour').size()
axes[0].plot(hourly_trips.index, hourly_trips.values, marker='o', linewidth=2)
axes[0].set_title('Trips by Hour of Day')
axes[0].set_xlabel('Hour')
axes[0].set_ylabel('Number of Trips')
axes[0].set_xticks(range(0, 24, 2))

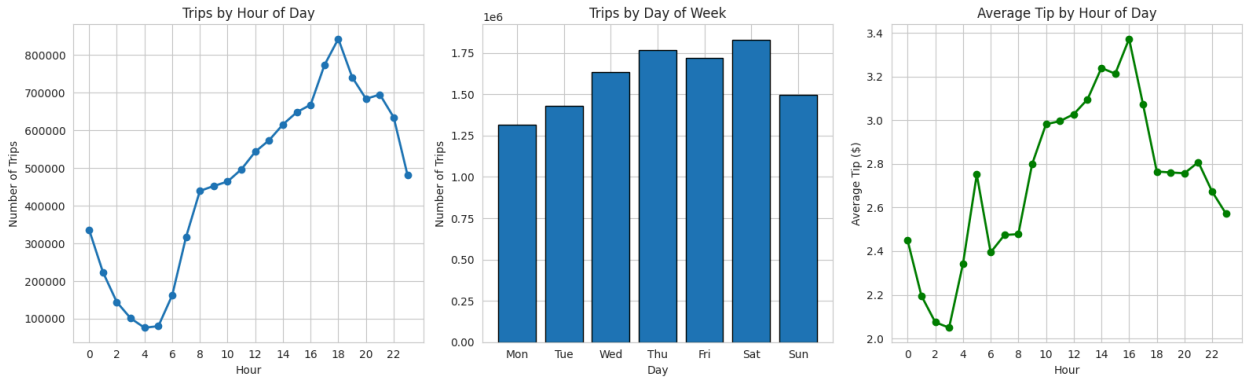
day_names = ['Mon', 'Tue', 'Wed', 'Thu', 'Fri', 'Sat', 'Sun']

daily_trips = eda.groupby('pickup_day').size()
axes[1].bar(range(7), [daily_trips.get(i, 0) for i in range(7)], edgecolor='black')
axes[1].set_title('Trips by Day of Week')
axes[1].set_xlabel('Day')
axes[1].set_ylabel('Number of Trips')
axes[1].set_xticks(range(7))
axes[1].set_xticklabels(day_names)

hourly_tips = eda.groupby('pickup_hour')['tip_amount'].mean()
axes[2].plot(hourly_tips.index, hourly_tips.values, marker='o', linewidth=2, color='green')
axes[2].set_title('Average Tip by Hour of Day')
axes[2].set_xlabel('Hour')
axes[2].set_ylabel('Average Tip ($)')
axes[2].set_xticks(range(0, 24, 2))
plt.tight_layout()
plt.show()
```

25]

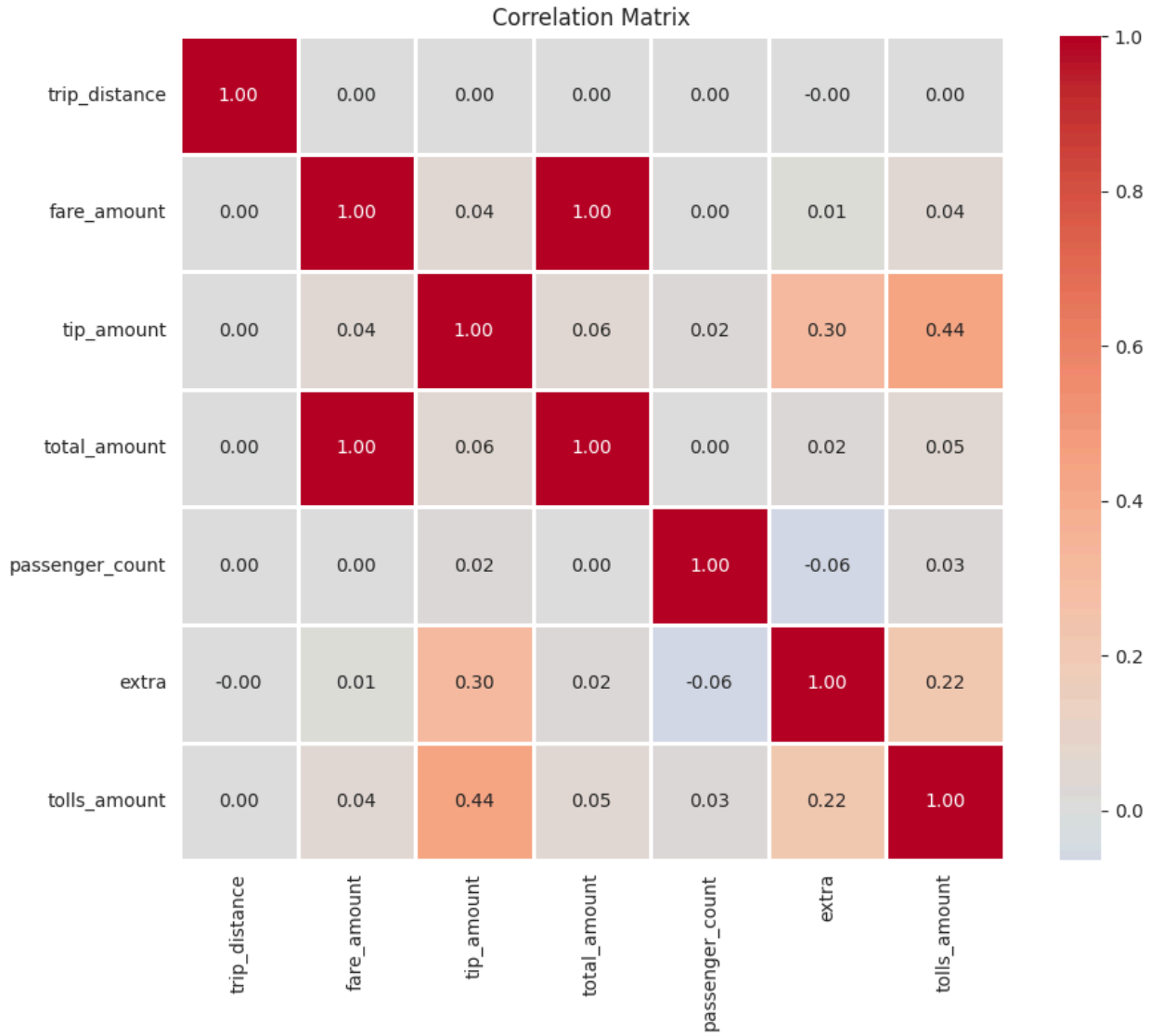
Datetime charts code



Datetime charts output || Most trips happen at around 6pm, on Thu/Fri/Sat and the average tip is highest at 4pm

```python
# Correlation matrix
correlation_cols = ['trip_distance', 'fare_amount', 'tip_amount', 'total_amount', 'passenger_count', 'extra', 'tolls_amount']
corr_matrix = eda[correlation_cols].corr()
plt.figure(figsize=(10, 8))
sns.heatmap(corr_matrix, annot=True, fmt='.2f', cmap='coolwarm', center=0, square=True, linewidths=1)
plt.title('Correlation Matrix')
plt.tight_layout()
plt.show()
```

Correlation matrix code



Correlation matrix output || Not a lot of multicollinearity, medium correlation between tips and extra fees/tolls amount might be artificial.

Appendix B

This appendix contains snippets of the initial preprocessing stage (preprocessing.ipynb)

```python
import os

java_path = os.getenv("JAVA_HOME")
print(f"Java Home: {java_path}")
print(f"Path exists: {os.path.exists(java_path) if java_path else False}")

cores = os.getenv("SPARK_CORES", "8")

from pyspark.sql import SparkSession

spark = (
    SparkSession.builder
    .appName("taxi_preprocessing")
    .master(f"local[{cores}]")
    .config("spark.driver.memory", "4g")
    .config("spark.driver.extraJavaOptions", "-Dio.netty.tryReflectionSetAccessible=true")
    .config("spark.sql.shuffle.partitions", int(cores) * 3)
    .getOrCreate()
)

spark.sparkContext.setLogLevel("WARN")
print("Spark version:", spark.version)
```

```
Java Home: C:\Program Files\Java\jdk-17
Path exists: True
Spark version: 3.5.1
```

Spark initialization and environment variables

```python
import glob

data_directory = "../data"

search_pattern = os.path.join(data_directory, "yellow_tripdata*.parquet")
data_paths = glob.glob(search_pattern)
data_paths.sort()

print(f"Found {len(data_paths)} files in Docker environment:")
for path in data_paths:
    print(f"  - {path}")
```

```
Found 3 files in Docker environment:
  - ../data\yellow_tripdata_2025-01.parquet
  - ../data\yellow_tripdata_2025-02.parquet
  - ../data\yellow_tripdata_2025-03.parquet
```

Files data paths

```
[ ]  df = spark.read.parquet(*data_paths)

     df.printSchema()
     df.show(5, truncate=True)
```

```
root
 |-- VendorID: integer (nullable = true)
 |-- tpep_pickup_datetime: timestamp_ntz (nullable = true)
 |-- tpep_dropoff_datetime: timestamp_ntz (nullable = true)
 |-- passenger_count: long (nullable = true)
 |-- trip_distance: double (nullable = true)
 |-- RatecodeID: long (nullable = true)
 |-- store_and_fwd_flag: string (nullable = true)
 |-- PULocationID: integer (nullable = true)
 |-- DOLocationID: integer (nullable = true)
 |-- payment_type: long (nullable = true)
 |-- fare_amount: double (nullable = true)
 |-- extra: double (nullable = true)
 |-- mta_tax: double (nullable = true)
 |-- tip_amount: double (nullable = true)
 |-- tolls_amount: double (nullable = true)
 |-- improvement_surcharge: double (nullable = true)
 |-- total_amount: double (nullable = true)
 |-- congestion_surcharge: double (nullable = true)
 |-- Airport_fee: double (nullable = true)
 |-- cbd_congestion_fee: double (nullable = true)

+--------+--------------------+--------------------+---------------+-------------+----------+-----------------
|VendorID|tpep_pickup_datetime|tpep_dropoff_datetime|passenger_count|trip_distance|RatecodeID|store_and_fwd_fla
+--------+--------------------+--------------------+---------------+-------------+----------+-----------------
...
|2       |2025-01-01 00:21:34 |2025-01-01 00:25:06  |3              |0.66         |1         |N
+--------+--------------------+--------------------+---------------+-------------+----------+-----------------
only showing top 5 rows
```

Reading and combining all files into one Spark frame

```python
df2 = (
    df.dropna()
    .withColumn(
        "trip_duration_min",
        (F.unix_timestamp("tpep_dropoff_datetime") - F.unix_timestamp("tpep_pickup_datetime")) / 60.0
    )
    .withColumn(
        "store_and_fwd_flag",
        F.when(F.col("store_and_fwd_flag") == "Y", 1).otherwise(0)
    )
    .drop("tpep_dropoff_datetime")
    .filter(F.col("trip_duration_min").between(1, 180))
    .filter(F.col("trip_distance").between(0.1, 300))
    .filter(F.col("tolls_amount") < 300)
    .filter(F.col("passenger_count").between(1, 6))
    .filter(F.col("RatecodeID") != 99.0)
    .filter(~F.col("payment_type").isin([2, 4, 5]))
    .filter( # for making sure all is positive (domain)
        (F.col("tip_amount") >= 0) &
        (F.col("mta_tax") >= 0) &
        (F.col("improvement_surcharge") >= 0) &
        (F.col("Airport_fee") >= 0) &
        (F.col("cbd_congestion_fee") >= 0) &
        (F.col("congestion_surcharge") >= 0) &
        (F.col("extra") >= 0))
)

print(f"Size of data after filtering: {df2.count()}")
df2.show(5, truncate=True)
```

[ ]

```
Size of data after filtering: 7247121
+--------+--------------------+---------------+-------------+----------+------------------+------------+------------+--------
|VendorID|tpep_pickup_datetime|passenger_count|trip_distance|RatecodeID|store_and_fwd_flag|PULocationID|DOLocationID|payment_
+--------+--------------------+---------------+-------------+----------+------------------+------------+------------+--------
|1       |2025-01-01 00:18:38 |1              |1.6          |1         |0                 |229         |237         |1
|1       |2025-01-01 00:32:40 |1              |0.5          |1         |0                 |236         |237         |1
|1       |2025-01-01 00:44:04 |1              |0.6          |1         |0                 |141         |141         |1
|2       |2025-01-01 00:33:58 |1              |0.56         |1         |0                 |263         |236         |1
|1       |2025-01-01 00:39:55 |1              |3.2          |1         |0                 |141         |113         |1
```

First step of preprocessing (cleaning, filtering) using Spark and Functions (SparkSQL)

```
[20]
df3 = (
    df2
    .withColumn("pickup_hour", F.hour("tpep_pickup_datetime"))
    .withColumn("pickup_dayofweek", F.dayofweek("tpep_pickup_datetime"))
    .withColumn("pickup_month", F.month("tpep_pickup_datetime"))
    .withColumn("is_weekend", F.when(F.col("pickup_dayofweek").isin([1,7]), 1).otherwise(0))
    .drop("tpep_pickup_datetime")
    .drop("payment_type") # drop payment type because its mostly credit card now anyways
    .drop("total_amount") # contains info about tip
    .drop("improvement_surcharge") # from EDA, a fixed fee with 98% imbalance
    .drop("mta_tax") # from EDA, same as improvement_surcharge logic
)
```

```
[21]
from pyspark.sql.functions import corr

df3.select(
    corr("fare_amount", "trip_distance").alias("fare_distance"),
    corr("fare_amount", "trip_duration_min").alias("fare_duration"),
    corr("fare_amount", "tip_amount").alias("fare_tip")
).show()
```

```
+------------------+------------------+------------------+
|     fare_distance|     fare_duration|          fare_tip|
+------------------+------------------+------------------+
|0.30437481303004005|0.27587997809951803|0.2436713878658857|
+------------------+------------------+------------------+
```

Second step of preprocessing (feature creation, dropping columns) and fare amount correlations

```
[ ]  df3.show(n=5, truncate=True)
```

```
+--------+---------------+-------------+----------+------------------+------------+------------+-----
|VendorID|passenger_count|trip_distance|RatecodeID|store_and_fwd_flag|PULocationID|DOLocationID|fare_
+--------+---------------+-------------+----------+------------------+------------+------------+-----
|1       |1              |1.6          |1         |0                 |229         |237         |10.0
|1       |1              |0.5          |1         |0                 |236         |237         |5.1
|1       |1              |0.6          |1         |0                 |141         |141         |5.1
|2       |1              |0.56         |1         |0                 |263         |236         |5.8
|1       |1              |3.2          |1         |0                 |141         |113         |26.1
+--------+---------------+-------------+----------+------------------+------------+------------+-----
only showing top 5 rows
```

```
[12]  df3.dtypes
```

```
[('VendorID', 'int'),
 ('passenger_count', 'bigint'),
 ('trip_distance', 'double'),
 ('RatecodeID', 'bigint'),
 ('store_and_fwd_flag', 'int'),
 ('PULocationID', 'int'),
 ('DOLocationID', 'int'),
 ('fare_amount', 'double'),
 ('extra', 'double'),
 ('tip_amount', 'double'),
 ('tolls_amount', 'double'),
 ('congestion_surcharge', 'double'),
 ('Airport_fee', 'double'),
 ('cbd_congestion_fee', 'double'),
 ('trip_duration_min', 'double'),
 ('pickup_hour', 'int'),
 ('pickup_dayofweek', 'int'),
 ('pickup_month', 'int'),
 ('is_weekend', 'int')]
```

Some information about our cleaned frame

```python
from pyspark.ml.regression import LinearRegression
from pyspark.ml.feature import VectorAssembler

feature_cols = [col for col in df3.columns if col != 'tip_amount']
assembler = VectorAssembler(inputCols=feature_cols, outputCol="features")
df_assembled = assembler.transform(df3)

# Lasso regression (L1 regularization)
lasso = LinearRegression(
    featuresCol="features",
    labelCol="tip_amount",
    elasticNetParam=1.0,
    regParam=0.01
)

lasso_model = lasso.fit(df_assembled)

coefficients = lasso_model.coefficients.toArray()

feature_importance = list(zip(feature_cols, coefficients))
selected_features = [(name, coef) for name, coef in feature_importance if abs(coef) > 0]

print("Selected features (non-zero coefficients):")
for name, coef in sorted(selected_features, key=lambda x: abs(x[1]), reverse=True):
    print(f"{name}: {coef:.4f}")

selected_feature_names = [name for name, coef in selected_features]
print(f"\nTotal features selected: {len(selected_feature_names)}")
```

```
Selected features (non-zero coefficients):
RatecodeID: 0.6444
VendorID: 0.6390
trip_distance: 0.3842
tolls_amount: 0.2281
extra: 0.2072
cbd_congestion_fee: 0.1113
trip_duration_min: 0.0788
is_weekend: -0.0268
Airport_fee: 0.0209
pickup_month: -0.0050
passenger_count: -0.0044
pickup_dayofweek: -0.0007
fare_amount: 0.0003

Total features selected: 13
```

Feature engineering using Lasso regularization

```python
from pyspark.ml.regression import DecisionTreeRegressor
from pyspark.ml.feature import VectorAssembler

# Assemble features
feature_cols = [col for col in df3.columns if col != 'tip_amount']
assembler = VectorAssembler(inputCols=feature_cols, outputCol="features")
df_assembled = assembler.transform(df3)

# Train Decision Tree to get feature importances
dt = DecisionTreeRegressor(
    featuresCol="features",
    labelCol="tip_amount",
    maxDepth=10,
    seed=42
)

dt_model = dt.fit(df_assembled)

# Get feature importances
importances = dt_model.featureImportances.toArray()

# Create feature importance list
feature_importance = list(zip(feature_cols, importances))

# Sort by importance
feature_importance_sorted = sorted(feature_importance, key=lambda x: x[1], reverse=True)

print("Feature Importances (Decision Tree):")
for name, importance in feature_importance_sorted:
    print(f"{name}: {importance:.4f}")

# Select top N features
top_n = 10
selected_feature_names = [name for name, imp in feature_importance_sorted[:top_n]]

print(f"\nTop {top_n} features: {selected_feature_names}")
```

```
Feature Importances (Decision Tree):
trip_distance: 0.8778
trip_duration_min: 0.0463
RatecodeID: 0.0452
tolls_amount: 0.0169
extra: 0.0030
pickup_hour: 0.0028
DOLocationID: 0.0022
PULocationID: 0.0018
cbd_congestion_fee: 0.0009
congestion_surcharge: 0.0008
passenger_count: 0.0005
is_weekend: 0.0005
Airport_fee: 0.0004
pickup_dayofweek: 0.0004
VendorID: 0.0003
pickup_month: 0.0001
store_and_fwd_flag: 0.0000

Top 10 features: ['trip_distance', 'trip_duration_min', 'RatecodeID', 'tolls_amount', 'extra', 'pickup_hour', 'DOLocationID', 'PULocationID', 'cbd_congestion_fee', 'congestion_surcharge']
```

Generate + Code + Markdown

Feature engineering using Decision Tree feature importance

Fairly big difference in the features that were selected. The actual feature selection will happen in the model Pipeline, to avoid data leakage.

```
df_final = df3
output_path = "../data/taxi_data_final"
df_final.write.mode("overwrite").parquet(output_path)

print("Data saved successfully using Spark!")

# 3. Print stats (Spark will re-calculate these efficiently)
print(f"Total rows: {df_final.count()}")
print(f"Total columns: {len(df_final.columns)}")
```

```
Data saved successfully!
Total rows: 7247121
Total columns: 19
```

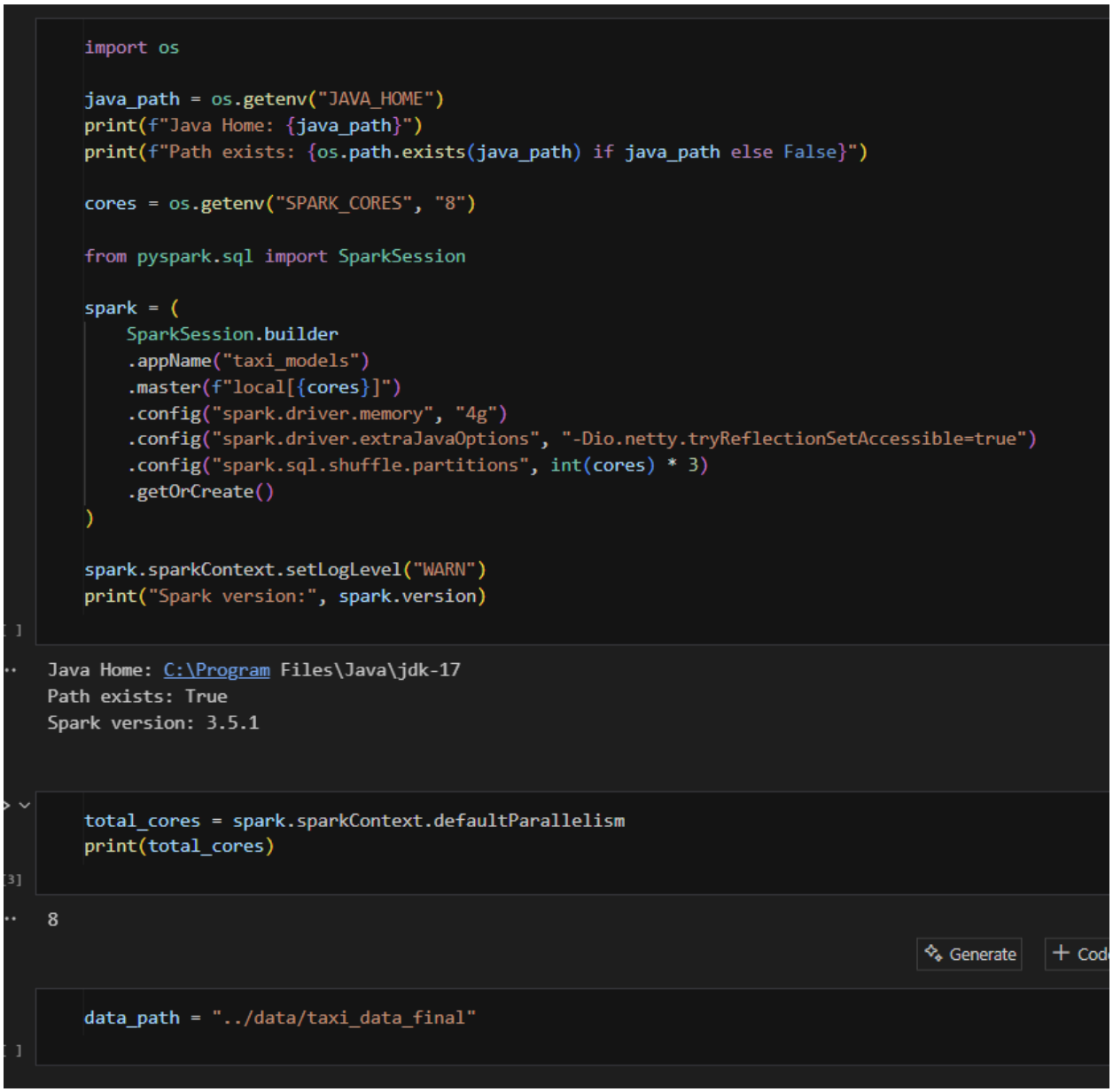
```
df_final.printSchema()
```

```
VendorID                int32
passenger_count         int64
trip_distance         float64
RatecodeID              int64
store_and_fwd_flag      int32
PULocationID            int32
DOLocationID            int32
fare_amount           float64
extra                 float64
tip_amount            float64
tolls_amount          float64
congestion_surcharge  float64
Airport_fee           float64
cbd_congestion_fee    float64
trip_duration_min     float64
pickup_hour             int32
pickup_dayofweek        int32
pickup_month            int32
is_weekend              int32
dtype: object
```

Saving our cleaned frame and last validation check

Appendix C

This appendix contains snippets of the model training (models.ipynb)



```
import os

java_path = os.getenv("JAVA_HOME")
print(f"Java Home: {java_path}")
print(f"Path exists: {os.path.exists(java_path) if java_path else False}")

cores = os.getenv("SPARK_CORES", "8")

from pyspark.sql import SparkSession

spark = (
    SparkSession.builder
    .appName("taxi_models")
    .master(f"local[{cores}]")
    .config("spark.driver.memory", "4g")
    .config("spark.driver.extraJavaOptions", "-Dio.netty.tryReflectionSetAccessible=true")
    .config("spark.sql.shuffle.partitions", int(cores) * 3)
    .getOrCreate()
)

spark.sparkContext.setLogLevel("WARN")
print("Spark version:", spark.version)
```

```
Java Home: C:\Program Files\Java\jdk-17
Path exists: True
Spark version: 3.5.1
```

```
total_cores = spark.sparkContext.defaultParallelism
print(total_cores)
```

```
8
```

Generate

```
data_path = "../data/taxi_data_final"
```

Spark initialization and validating core numbers and data path

```
df = spark.read.parquet(data_path)

df.show(n=5, truncate=True)
```

```
+--------+---------------+-------------+----------+------------------+------------
|VendorID|passenger_count|trip_distance|RatecodeID|store_and_fwd_flag|PULocationID
+--------+---------------+-------------+----------+------------------+------------
|1       |1              |1.6          |1         |0                 |229
|1       |1              |0.5          |1         |0                 |236
|1       |1              |0.6          |1         |0                 |141
|2       |1              |0.56         |1         |0                 |263
|1       |1              |3.2          |1         |0                 |141
+--------+---------------+-------------+----------+------------------+------------
only showing top 5 rows
```

```
df.printSchema()
```

```
root
 |-- VendorID: integer (nullable = true)
 |-- passenger_count: long (nullable = true)
 |-- trip_distance: double (nullable = true)
 |-- RatecodeID: long (nullable = true)
 |-- store_and_fwd_flag: integer (nullable = true)
 |-- PULocationID: integer (nullable = true)
 |-- DOLocationID: integer (nullable = true)
 |-- fare_amount: double (nullable = true)
 |-- extra: double (nullable = true)
 |-- tip_amount: double (nullable = true)
 |-- tolls_amount: double (nullable = true)
 |-- congestion_surcharge: double (nullable = true)
 |-- Airport_fee: double (nullable = true)
 |-- cbd_congestion_fee: double (nullable = true)
 |-- trip_duration_min: double (nullable = true)
 |-- pickup_hour: integer (nullable = true)
 |-- pickup_dayofweek: integer (nullable = true)
 |-- pickup_month: integer (nullable = true)
 |-- is_weekend: integer (nullable = true)
```

Info about our frame

```
from pyspark.ml import Pipeline
from pyspark.ml.feature import StringIndexer, OneHotEncoder, VectorAssembler, StandardScaler
from pyspark.ml.evaluation import RegressionEvaluator

train, test = df.randomSplit([0.8, 0.2], seed=42)

train.cache()
test.cache()
train.count()
test.count()

all_features = [col for col in df.columns if col != "tip_amount"]
categorical_features = ["VendorID", "RatecodeID"]
numerical_features = [col for col in all_features if col not in categorical_features]
```

```
numerical_features
```

```
['passenger_count',
 'trip_distance',
 'store_and_fwd_flag',
 'PULocationID',
 'DOLocationID',
 'fare_amount',
 'extra',
 'tolls_amount',
 'congestion_surcharge',
 'Airport_fee',
 'cbd_congestion_fee',
 'trip_duration_min',
 'pickup_hour',
 'pickup_dayofweek',
 'pickup_month',
 'is_weekend']
```

Splitting our data 80/20, caching results (using count because of lazy evaluation) and dividing our numerical and categorical features

```python
# Check sizes
print(f"Train: {train.count()}, Test: {test.count()}")

# Check a few tip_amount values
train.select("tip_amount").show(5)
test.select("tip_amount").show(5)
```

```
Train: 5797352, Test: 1449769
+----------+
|tip_amount|
+----------+
|       2.0|
|      12.0|
|      2.05|
|       1.0|
|       2.5|
+----------+
only showing top 5 rows

+----------+
|tip_amount|
+----------+
|       3.0|
|       3.0|
|       2.0|
|       0.0|
|       0.0|
+----------+
only showing top 5 rows
```

Validating train and test sets

```python
stages_prepr = [] # preprocessing pipeline stage

for cat_col in categorical_features:
    indexer = StringIndexer(inputCol=cat_col, outputCol=f"{cat_col}_idx", handleInvalid="keep")
    encoder = OneHotEncoder(inputCol=f"{cat_col}_idx", outputCol=f"{cat_col}_ohe")
    stages_prepr += [indexer, encoder]

assembler_inputs = numerical_features + [f"{cat}_ohe" for cat in categorical_features]
assembler = VectorAssembler(inputCols=assembler_inputs, outputCol="features")
stages_prepr.append(assembler)
```

```python
# Evaluators
evaluator_rmse = RegressionEvaluator(labelCol="tip_amount", predictionCol="prediction", metricName="rmse")
evaluator_mae = RegressionEvaluator(labelCol="tip_amount", predictionCol="prediction", metricName="mae")
evaluator_r2 = RegressionEvaluator(labelCol="tip_amount", predictionCol="prediction", metricName="r2")
```

```python
from pyspark.ml.regression import LinearRegression

scaler = StandardScaler(inputCol="features", outputCol="features_scaled", withMean=True, withStd=True)
lr = LinearRegression(featuresCol="features_scaled", labelCol="tip_amount", regParam=0.1, elasticNetParam=0.5)

lr_pipeline = Pipeline(stages=stages_prepr + [scaler, lr])

lr_model = lr_pipeline.fit(train)
```

Setting shared pipeline preprocessing stages among all models (stages_prepr), creating shared evaluators and creating the Linear Regression pipeline

```
lr_test_predictions = lr_model.transform(test)
lr_train_predictions = lr_model.transform(train)
train_r2 = evaluator_r2.evaluate(lr_train_predictions)
test_rmse = evaluator_rmse.evaluate(lr_test_predictions)
test_mae = evaluator_mae.evaluate(lr_test_predictions)
test_r2 = evaluator_r2.evaluate(lr_test_predictions)

print(f">> Train R2: {train_r2:.4f}")

print("\nTest Metrics:")
print(f">> RMSE: {test_rmse:.4f}")
print(f">> MAE: {test_mae:.4f}")
print(f">> R2: {test_r2:.4f}")
```

```
>> Train R2: 0.6018

Test Metrics:
>> RMSE: 2.4048
>> MAE: 1.2783
>> R2: 0.6041
```

Evaluating the Linear Regression models (different results then the report because of difference in Spark Cores)

```
from pyspark.ml.regression import DecisionTreeRegressor

dt = DecisionTreeRegressor(featuresCol="features", labelCol="tip_amount")
dt_pipeline = Pipeline(stages=stages_prepr + [dt])
```

Setting the Decision Tree pipeline

```python
import itertools
from pyspark.sql import functions as F

def grid_search_cv(df, pipeline, param_options, k=5, seed=42):
    # generate param grid as all combinations
    keys = param_options.keys()
    values = param_options.values()
    param_grid = [dict(zip(keys, v)) for v in itertools.product(*values)]

    print(f"Total combinations to test: {len(param_grid)}")

    # generate fold ids and cache data
    df_with_folds = df.withColumn("fold_id", (F.rand(seed) * k).cast("int")).cache()

    best_rmse = float("inf")
    best_params = None
    best_model = None

    # loop through parameted grid
    for params in param_grid:
        print(f"\nTesting parameters: {params}")
        fold_scores = []

        # kfold cv
        for i in range(k):
            train_fold = df_with_folds.filter(F.col("fold_id") != i)
            val_fold = df_with_folds.filter(F.col("fold_id") == i)

            # access dt (last stage of our pipeline)
            dt_stage = pipeline.getStages()[-1]
            for param_name, value in params.items():
                dt_stage.set(dt_stage.getParam(param_name), value)

            model = pipeline.fit(train_fold)
            predictions = model.transform(val_fold)
            rmse = evaluator_rmse.evaluate(predictions)
            r2 = evaluator_r2.evaluate(predictions)
            fold_scores.append(rmse)
            print(f">> Fold {i+1} RMSE: {rmse:.4f} || R2: {r2:.4f}")

        avg_rmse = sum(fold_scores) / len(fold_scores)
        print(f">> Average RMSE: {avg_rmse:.4f}")
```

Part 1 of our cross validator (Grid Search/KFold)

```
            # track best model params
            if avg_rmse < best_rmse:
                best_rmse = avg_rmse
                best_params = params

    print(f"\nBest model: {best_params} with Avg RMSE: {best_rmse:.4f}")

    # train best model
    final_dt_stage = pipeline.getStages()[-1]
    for param_name, value in best_params.items():
        final_dt_stage.set(final_dt_stage.getParam(param_name), value)

    best_model = pipeline.fit(df)

    df_with_folds.unpersist() # uncache since we wont use it anymore
    return best_model

params = {
    "maxDepth": [5, 10, 15],
    "maxBins": [32, 64]
}

dt_model = grid_search_cv(train, dt_pipeline, params, k=5)
```

Part 2 of our cross validator (Grid Search/KFold)

```
Total combinations to test: 6

Testing parameters: {'maxDepth': 5, 'maxBins': 32}
>> Fold 1 RMSE: 2.3497 || R2: 0.6146
>> Fold 2 RMSE: 2.4124 || R2: 0.6049
>> Fold 3 RMSE: 2.3803 || R2: 0.6089
>> Fold 4 RMSE: 2.3951 || R2: 0.6062
>> Fold 5 RMSE: 2.3791 || R2: 0.6084
>> Average RMSE: 2.3833

Testing parameters: {'maxDepth': 5, 'maxBins': 64}
>> Fold 1 RMSE: 2.3460 || R2: 0.6158
>> Fold 2 RMSE: 2.4130 || R2: 0.6047
>> Fold 3 RMSE: 2.3821 || R2: 0.6083
>> Fold 4 RMSE: 2.3941 || R2: 0.6066
>> Fold 5 RMSE: 2.3801 || R2: 0.6081
>> Average RMSE: 2.3831

Testing parameters: {'maxDepth': 10, 'maxBins': 32}
>> Fold 1 RMSE: 2.3421 || R2: 0.6171
>> Fold 2 RMSE: 2.4132 || R2: 0.6046
>> Fold 3 RMSE: 2.4131 || R2: 0.5980
>> Fold 4 RMSE: 2.3975 || R2: 0.6054
>> Fold 5 RMSE: 2.3784 || R2: 0.6086
>> Average RMSE: 2.3889
...
>> Fold 5 RMSE: 2.5002 || R2: 0.5675
>> Average RMSE: 2.5125

Best model: {'maxDepth': 10, 'maxBins': 64} with Avg RMSE: 2.3812
```

Snippet of the cross validator output

```python
dt_test_predictions = dt_model.transform(test)
dt_train_predictions = dt_model.transform(train)

train_r2 = evaluator_r2.evaluate(dt_train_predictions)
test_rmse = evaluator_rmse.evaluate(dt_test_predictions)
test_mae = evaluator_mae.evaluate(dt_test_predictions)
test_r2 = evaluator_r2.evaluate(dt_test_predictions)

print(f"Best parameters: maxDepth={dt_model.stages[-1].getMaxDepth()}, maxBins={dt_model.stages[-1].getMaxBins()}")

print(f"\n>> Train R2: {train_r2:.4f}")

print("\nTest Metrics:")
print(f">> Test RMSE: {test_rmse:.4f}")
print(f">> Test MAE: {test_mae:.4f}")
print(f">> Test R2: {test_r2:.4f}")
```

```
[Stage 1096:=========================================
Best parameters: maxDepth=10, maxBins=64

>> Train R2: 0.6192

Test Metrics:
>> Test RMSE: 2.3744
>> Test MAE: 1.2603
>> Test R2: 0.6106
```

Evaluating the Decision Tree model (used the JupyterLab results here instead of VSCode)

```python
# threading might make it slower, so we will just leave Spark do its parallelization, even though the training of models is technically sequential

def bootstrapping(df, pipeline, num_trees=5, params={}):
    train_df = df
    train_df.cache()

    models = []

    for i in range(num_trees):
        print(f"Training Tree {i+1}/{num_trees}")

        # bootstrapping
        bootstrap_sample = train_df.sample(withReplacement=True, fraction=1.0, seed=i+42)

        dt_stage = pipeline.getStages()[-1]
        for param_name, value in params.items():
            dt_stage.set(dt_stage.getParam(param_name), value)

        model = pipeline.fit(bootstrap_sample)
        models.append(model)

    train_df.unpersist()

    return models

params = {
    "maxDepth": 10,
    "maxBins": 64,
    "minInstancesPerNode": 1
}

bootstraps = bootstrapping(df=train, pipeline=dt_pipeline, num_trees=10, params=params)
```

```
Training Tree 1/10
Training Tree 2/10
Training Tree 3/10
Training Tree 4/10
Training Tree 5/10
Training Tree 6/10
Training Tree 7/10
Training Tree 8/10
Training Tree 9/10
Training Tree 10/10
```

Bootstrapping (part 1 of Bagging Ensemble)

```python
def aggregating_predictions(models, test_df):
    # id for joins and cache
    test_with_id = test_df.withColumn("row_id", F.monotonically_increasing_id()).cache()

    #  only ids and target
    final_df = test_with_id.select("row_id", "tip_amount")

    pred_cols = []

    for i, model in enumerate(models):
        col_name = f"tree_pred_{i}"
        pred_cols.append(col_name)

        print(f"Adding predictions from Tree {i}")
        single_tree_pred = model.transform(test_with_id).select("row_id", F.col("prediction").alias(col_name))
        final_df = final_df.join(single_tree_pred, on="row_id", how="inner")

    # aggregation
    final_df = final_df.withColumn(
        "prediction",
        sum(F.col(c) for c in pred_cols) / len(models)
    )

    result = final_df.join(test_with_id.drop("tip_amount"), on="row_id").drop("row_id").select("tip_amount", "prediction")

    test_with_id.unpersist()

    return result

ensemble = aggregating_predictions(bootstraps, test)
print(f"Rows after ensemble: {ensemble.count()}")
```

[22]

```
Adding predictions from Tree 0
Adding predictions from Tree 1
Adding predictions from Tree 2
Adding predictions from Tree 3
Adding predictions from Tree 4
Adding predictions from Tree 5
Adding predictions from Tree 6
Adding predictions from Tree 7
Adding predictions from Tree 8
Adding predictions from Tree 9
Rows after ensemble: 1449769
```

Aggregation (part 2 of Bagging Ensemble)

```
[23]
ensemble.show(5)
```

```
+----------+------------------+
|tip_amount|        prediction|
+----------+------------------+
|       0.0|2.2320381990221847|
|       1.9|1.7676512484906273|
|       2.2|2.2320381990221847|
|     13.55|10.585844050829428|
|       2.2|1.9817743639407293|
+----------+------------------+
only showing top 5 rows
```

```
[24]
ensemble_rmse = evaluator_rmse.evaluate(ensemble)
ensemble_mae = evaluator_mae.evaluate(ensemble)
ensemble_r2 = evaluator_r2.evaluate(ensemble)

print(f">> Ensemble RMSE: {ensemble_rmse:.4f}")
print(f">> Ensemble MAE: {ensemble_mae:.4f}")
print(f">> Ensemble R2: {ensemble_r2:.4f}")
```

```
>> Ensemble RMSE: 2.3622
>> Ensemble MAE: 1.2593
>> Ensemble R2: 0.6180
```

Ensemble did improve our metrics but ever so slightly. This signifies that to improve our model we might nee
example, getting new features, especially the human behavior once, would aid greatly.

```
[25]
spark.catalog.clearCache()
```

Ensemble evaluation and clearing the cache

Appendix D

This appendix contains snippets of the Dockerfile and requirements.txt

```dockerfile
FROM python:3.11-slim

RUN apt-get update && \
    apt-get install -y --no-install-recommends openjdk-21-jre-headless procps && \
    apt-get clean && rm -rf /var/lib/apt/lists/*

ENV JAVA_HOME=/usr/lib/jvm/java-21-openjdk-amd64
ENV PATH=$PATH:$JAVA_HOME/bin

WORKDIR /app

COPY requirements.txt .
RUN pip install --no-cache-dir -r requirements.txt

COPY data/ ./data/
COPY scripts/ ./scripts/
COPY README.md ./README.md

ENV SPARK_HOME=/usr/local/lib/python3.11/site-packages/pyspark
ENV PATH=$PATH:$SPARK_HOME/bin
ENV SPARK_CORES=8

EXPOSE 8888 4040

CMD ["jupyter", "lab", "--ip=0.0.0.0", "--port=8888", "--no-browser", "--allow-root"]
```

Dockerfile, exposing ports 8888 and 4040 for browser access

```
numpy
matplotlib
pandas
seaborn
jupyterlab
pyarrow
pyspark==3.5.1
scikit-learn
```

Required packages for the Docker container